



Chapter – 5

User & Group Management

(Creation, Updating & Deletion)

Understanding Users and Groups

Linux is a multiuser system that relies on accounts—data structures and procedures used to identify individual users of a computer. Managing these accounts is a basic but important system administration skill. Before delving into the details, you need to understand a few basic concepts about user and group administration, which are covered in the following sections. Linux accounts are like accounts on Windows, Mac OS, and other OSs. Some websites use accounts too. Nonetheless, a few details deserve explanation. These include the various Linux users, the nature of Linux groups, and the way Linux maps the numbers it uses internally to the usernames and group names that people generally use.

Learning about Linux User Accounts

Typical Linux accounts are individual user accounts identified via the account's username. These accounts are for people who need access to the system. Linux accounts can also be accounts for system services called **daemons**. There can also be specialty accounts that are created for unique purposes. For example, you may want a user account to receive email but not be able to access the local system. Each account requires a unique username. If you can create accounts on your system with mixed-case usernames, Linux treats usernames in a **case-sensitive** way. Therefore, a single computer can support both ellen and Ellen as separate users. This practice can lead to a great deal of confusion, so it's best to avoid creating accounts whose usernames differ only in case.



Linking Users Together via Groups

Following are the three main user administration files:

`/etc/passwd`

Keeps user account and password information. This file holds the majority of information about accounts on the Linux / UNIX system.

`/etc/shadow`

Holds the encrypted password of the corresponding account. Not all Linux systems support this file.

`/etc/group`

This file contains the group information for each account.

`/etc/gshadow`

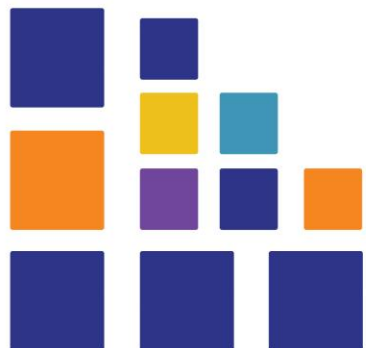
This file contains secure group account information.



Linking Users Together via Groups *continues...

Linux uses groups as a means of organizing users. In many ways, groups parallel users. In particular, they're defined in similar configuration files, have names similar to usernames, and are represented internally by numbers (as are accounts). Groups are not accounts, however. Rather, groups help organize collections of accounts, largely as a security measure. Every file on a Linux system is associated with a specific user and a specific group. Various permissions can be assigned to members of a **group**. For instance, group members (such as faculty at a university) may be allowed to read a file, but others (such as students) may be disallowed such access. Because Linux provides access to most hardware devices (such as scanners) through files, you can also use this same mechanism to control access to hardware.

Every group has anywhere from no members to as many members as there are users on the computer. Group membership is controlled through the `/etc/group` file. This file contains a list of groups and the members belonging to each group.



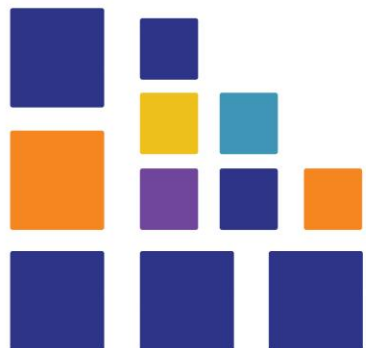
A user can access files belonging to other groups as long as the user belongs to that group and the group access permissions allow such access. To run programs or create files with a group other than the primary group, the user must run the **newgrp** command to switch current group membership. For instance, to change to the project2 group as your current group, you might type the following:

```
$ newgrp project2
```

If the user typing this command is listed as a project2 group member in the `/etc/group` file, the user's current group membership changes.

Mapping UIDs and GIDs to Users and Groups

As mentioned earlier, Linux defines users and groups by numbers, referred to as *user IDs (UIDs)* and *group IDs (GIDs)*, respectively. Internally, Linux tracks users and groups by these numbers, not by their name. Linux distributions reserve at least the first 100 user and group IDs (0–99) for system use. The most important of these is 0, which corresponds to root (both the user and the group). Accounts and groups that are associated with specific Linux utilities and functions use succeeding low numbers.



Configuring User Accounts

How frequently you perform user maintenance depends on the nature of the system you administer. Some systems, such as small personal workstations, rarely require changes. Others, such as multiuser servers, may require daily maintenance. The latter situation would seem to require more knowledge of user account configuration tools. Yet, even in a seldom-changing system, it's useful to know how to add, modify, or delete accounts so that you can do so quickly and correctly when needed.

Adding Users

Adding users can be accomplished through the `useradd` utility. (This program is called `adduser` on some distributions.) Its basic syntax is as follows:

```
useradd [-c comment] [-d home-dir] [-e expire-date] [-f inactive-days] [-g default-group] [-G group[,...]] [-m [-k skeleton-dir] | -M] [-p password] [-s shell] [-u UID [-o]] [-r] [-N] username
```

Example:

You want to create an account for a user named Sally in this directory and place her home directory on the new disk. You want to make the new user a member of the `project1` and `project4` groups, with default membership in `project4`. The following `useradd` command accomplishes this goal:

```
# useradd -m -d /home2/sally -g project4 -G project1,project4 sally
```

Modifying User Accounts

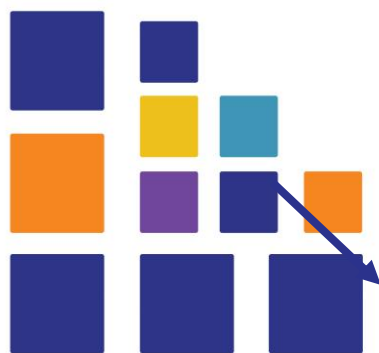
User accounts may be modified in many ways: You can directly edit critical files such as /etc/passwd (though not recommended), modify user-specific configuration files in the account's home directory, or use system utilities like those used to create accounts.

You usually modify an existing user's account at the user's request or to implement some new policy or system change, such as moving home directories to a new hard disk.

Setting a Password

Although useradd provides the -p parameter to set a password, this tool isn't very useful when directly adding a user because it requires a pre-hashed password. Therefore, it's usually easiest to create an account in disabled form (by not using -p with useradd) and set the password after creating the account. You can do this with the passwd command, which has the following syntax:

```
passwd [-k] [-l] [-u [-f]] [-d] [-S] [username]
```



Consider the previously created user account for Sally. In order for Sally to use her account, she must have the password set. It was not set using the useradd command. You can use the passwd command on her account as follows:

`# passwd sally`

```
Changing password for user sally
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully
```

Modify an Account:

The usermod command enables you to make changes to an existing account from the command line. It uses the same arguments as the useradd command, plus the -l argument, which allows you to change the account name.

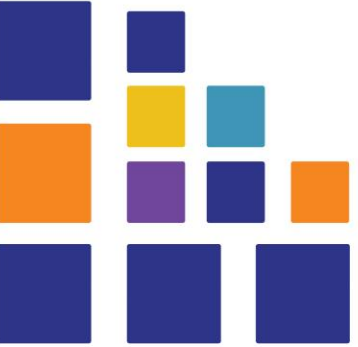
For example, to change the account name mcmohd to mcmohd20 and to change home directory accordingly, you would need to issue following command:

`$ usermod -d /home/mcmohd20 -m -l mcmohd mcmohd20`



Delete an Account:

- The `userdel` command can be used to delete an existing user. This is a very dangerous command if not used with caution. There is only one argument or option available for the command: `-r`, for removing the account's home directory and mail file.
- For example, to remove account `mcmohd20`, you would need to issue following command:
- `$ userdel -r mcmohd20`
- If you want to keep her home directory for backup purposes, omit the `-r` option. You can remove the home directory as needed at a later time.



To find all the uses files:

```
# find / -uid 1029
```

To delete the user account from Samba's database

```
smbpasswd -x sally
```

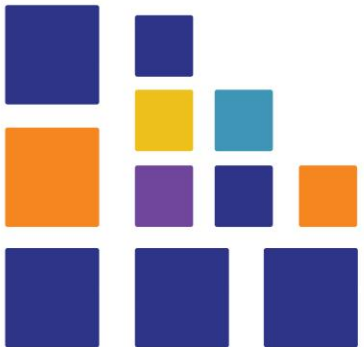
Using chage

The chage command allows you to modify account settings relating to account expiration. It's possible to configure Linux accounts so that they automatically expire if either of two conditions is true:

1. The password hasn't been changed in a specified time period.
2. The system date is past a predetermined time.

These settings are controlled through the chage utility, which has the following syntax:

```
chage [-l] [-m mindays] [-M maxdays] [-d lastday] [-I inactivedays] [-E expiredate] [-W warndays] username
```



Configuring group accounts

Linux provides group configuration tools that parallel those for user accounts in many ways. Groups are not accounts so many features of these tools vary. Their layout is similar to that for account control files, but the details differ.

Adding Groups

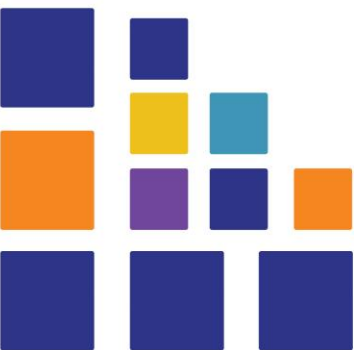
Linux provides the groupadd command to add a new group. This utility is similar to useradd but has fewer options. The groupadd syntax is as follows:

```
groupadd [-g GID [-o]] [-r] [-f] groupname
```

Option	Description
-g GID	The numerical value of the group's ID.
-o	This option permits to add group with non-unique GID
-r	This flag instructs groupadd to add a system account
-f	This option causes to just exit with success status if the specified group already exists. With -g, if specified GID already exists, other (unique) GID is chosen
groupname	Actual group name to be created.

If you do not specify any parameter then system would use default values.

Following example would create developers group with default values, which is very much acceptable for most of the administrators.



```
$ groupadd developers
```

Modifying Group Information

Group information, like user account information, may be modified either by using utility programs or by directly editing the underlying configuration file, `/etc/group`. There are fewer options for modifying groups than for modifying accounts, and the utilities and configuration files are similar.

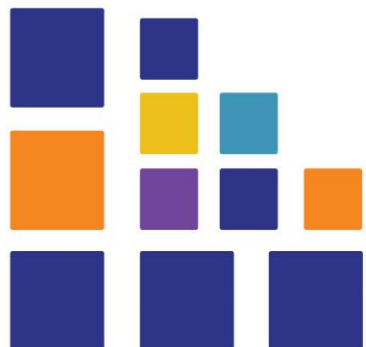
Using `groupmod`

The `groupmod` command modifies an existing group's settings. Its syntax is as follows:

```
groupmod [-g GID [-o]] [-n newgroupname] oldgroupname
```

The options to this command modify its operation:

Specify a GID Specify the new group ID using the `-g GID` option. The `groupmod` returns an error if you specify a new group ID that's already in use. Specify a Group Name Specify a new group name with the `-n newgroupname` option.



Using gpasswd

The gpasswd command is the group equivalent to passwd. The gpasswd command also enables you to modify other group features and to assign group administrators users who may perform some group-related administrative functions for their groups. The basic syntax for this command is as follows:

```
gpasswd [-a user] [-d user] [-R] [-r] [-A user,...] [-M user,...] group
```

The options for this command modify its actions:

Add a User The -a user option adds the specified user to the specified group.

Delete a User The -d user option deletes the specified user from the specified group.

Disallow newgrp Additions The -R option configures the group not to allow anybody to become a member through newgrp.

Remove Password The -r option removes the password from a group.

Add Group Administrators The root user may use the -A user[...] parameter to specify group administrators. Group administrators may add members to and remove members from a group and change the group password.

Add Users The -M user[...] option works like -A, but it also adds the specified user(s) to the list of group members. If entered without any parameters except a group name, gpasswd changes the password for the group.

Delete a Group:

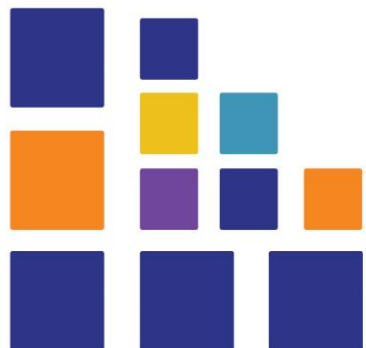
To delete an existing group, all you need are the `groupdel` command and the group name. To delete the financial group, the command is:

```
# groupdel developer
```

This removes only the group, not any files associated with that group. The files are still accessible by their owners.

you can find all of the files on the computer with that GID by using the following command:

```
# find / -gid 1003
```



Viewing Individual Account Records

When you're managing user accounts, groups, and passwords, individual account information often needs to be reviewed. However, it can be tedious to locate particular records in the various account management files. Often, system administrators will use the `grep` command to search through these files for particular records:

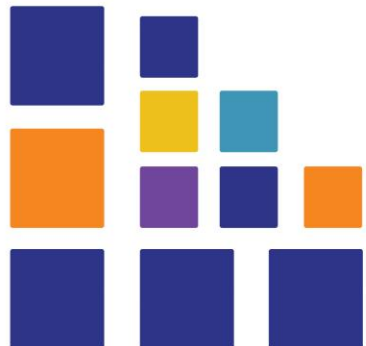
```
$ grep Christine /etc/passwd
```

```
Christine:x:501:501::/home/Christine:/bin/bash
```

Another method for record retrieval uses the `get` entry command, `getent`. The `getent` command allows you to access records stored in one of the system databases. Only databases configured in the Name Service Switch (NSS) file, `/etc/nsswitch.conf`, can be accessed.

The general syntax for this command is as follows:

```
getent database key
```



The database parameter can be any of the NSS configured system databases. For our purposes here, database can be one of the following files:

- passwd** (the `/etc/passwd` file)
- shadow** (the `/etc/shadow` file)
- group** (the `/etc/group` file)

you want to search the /etc/passwd file for the Christine user account:

```
$ getent passwd Christine
```

```
Christine:x:501:501::/home/Christine:/bin/bash
```

You may search only one database at a time, and only those with super user privileges can successfully search the /etc/shadow file with the getent command.

```
$ getent shadow Christine
```

The getent command is useful for viewing which users have membership in a particular group. In this case, the key parameter in the getent group command is the group's name:

```
$ getent group Production
```

```
Production:x:504:sally,Christine
```

when you use the user account name for key in the getent group command, only the user's primary group and GID is displayed:

```
$ getent group sally
```

```
sally:x:503:
```

To search for all a user's group memberships, use the groups command instead:

```
$ groups sally
```

```
sally : sally users Production project1
```


End of Chapter 5

